Module - 4

Inter-process communication

1. Explain Multiple processes in an application.

In computing, an application can have multiple processes running simultaneously to achieve various task.

Process :

A process is an instance of a program that is being executed. It has its own memory space and resources. processes allow applications to run different tasks independently.

Multiple processes in an application :

An application may require multiple processes to function efficiently. Each process can perform different tasks or split a large task into smaller ones. For example, a Web browser may run separate processes for each tab, so if one tab crashes, the others are unaffected.

How Multiple processes work Together :

Inter-process communication (IPC) :

Multiple processes can exchange information using various methods such as message passing, shared memory.

Concurrency :

Multiple processes can run simultaneously on multi-core processors, improving application performance by handling tasks in parallel.

Benefits of using Multiple Processes:

Fault Isolation: If one process crashes, it doesn't affect other processes.

Resource Management: Different processes can be assigned different levels of priority and system resources (CPU, memory).

Scalability: Applications can handle more tasks efficiently by distributing the work load across multiple processes.

Example:

Consider a video editing application:

→ One process may handle user interactions.
→ Another process may render video in the background
→ A third process might handle file input/output operations.

Using multiple processes in an application improves performance, reliability and scalability. Each process works independently but may communicate with other processor to ensure the application functions as a whole.

2. Explain Multiple Threads in an application.

Multiple threads in an application allow different parts of a program to run concurrently, improving performance and efficiency, especially on multi-core processors.

Thread :

A thread is the smallest unit of execution within a process. A process can have one or more threads, each executing a part of the program. All threads within a process share the same memory space and system resources, which allows them to communicate and work together efficiently.

Multi threading :

In a multithreaded application, multiple threads run simultaneously or nearly simultaneously. This allows the cpu to handle multiple operations at once, improving the applications performance, especially for tasks that can run independently.

Example of Multi threading :

Consider a video streaming application:

→ One thread handles downloading the video data.

→ Another thread decodes and renders the video for playback.

→ A third thread manages user interactions, like play/pause or adjusting settings.

→ By separating these tasks into different threads, the application can ensure smooth play back while still responding to user inputs.

Multithreading is a powerful way to enhance the performance and responsiveness of an application. By running multiple tasks concurrently, an application can use system resources more efficiently and

provide a better user experience. However, developers must manage threads carefully to avoid issues like race conditions, deadlocks, and increased complexity.

3. Explain the concept of tasks, task and thread states.

Tasks :-

A task is a unit of work being executed.

The task in operating system may be synonymous with the process. A task is a subpart of a job. Tasks combine to form a job.

The task in operating system may be synonymous with the process. A

The task is obscure in the sense that it has many meanings.

The task may be a thread, process, a single job, and much more.

A task is termed a thread when it is undergoing execution.

Example :-

When we run a thread in Java, it is called a task. If a printer prints a document, it is said to perform a printing task. More than one task can be performed together at the same time and it is known as multi tasking. Multitasking is also known as time-sharing. Multitasking is an extension of multiprogramming operating system.

→ In an operating system (OS), a thread is a light weight process that runs within a larger process or the OS, and thread states are the different conditions a thread can be in:

Threads :

A thread is a task within a process that has its own stack, program counter, and set of registers. A process can have multiple threads, and each thread belongs to only one process.

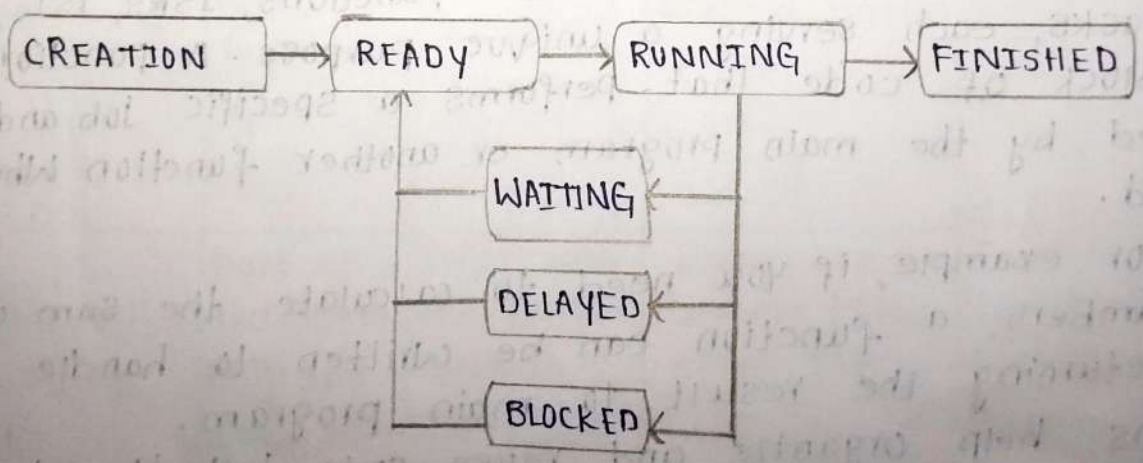
Thread states :

The states a thread can be in are new, runnable, blocked, and terminated.

Thread states in operating systems

When a thread moves through the system, it is always in one of the five states.

1. Ready
2. Running
3. Waiting
4. Delayed
5. Blocked.



→ When an application is to be processed, then it creates a thread.

→ It is then allocated the required resources (such as network) and it comes in the READY queue.

→ When the thread scheduler (like a process scheduler) assign the thread with processor, it comes it RUNNING queue.

→ When the process needs some other event to be triggered, which is outsides it's control, it transitions from RUNNING to WAITING queue.

→ When the application has the capability to delay the processing of the thread, it when needed can delay the thread and put it to sleep for a specific amount of time. The thread then transitions from RUNNING to DELAYED queue.

→ When thread generates an I/O request and cannot move further till its done, it transitions from RUNNING to BLOCKED queue.

→ After the process is completed, the thread transitions from RUNNING to FINISHED.

4. Distinction between function, ISR, IST and task by their characteristics.

In embedded systems, we use Functions, ISRs, ISTs, and TASKS, each serving a unique purpose. A Function is a block of code that performs a specific job and is called by the main program or another function when needed.

For example, if you need to calculate the sum of two numbers, a function can be written to handle this, returning the result to main program. Functions help organize and reuse code, but they don't have any special priority and just run in the main program flow.

Feature	Function	ISR (Interrupt Service Routine)	IST (Interrupt Service Task)	Task
Purpose	Perform a specific Job	Handle urgent hardware events	Do additional work after an ISR.	Manage ongoing or background activities.
When it Runs	called by the main program or another function	Automatically when an interrupt occurs.	After an ISR has finished handling an interrupt.	Scheduled by the operation system
Duration	Short and quick	Very short and fast.	can take longer since its not urgent	can run continuously or periodically
Example	Adding two numbers	Responding to a button press.	Processing sensor data after a button press.	Blinking an LED continuosly
How it Runs	Part of the normal program flow	Interrupts the current task to run.	Runs like a regular task after ISR	Runs independently, managed by OS.

→ An ISR (Interrupt service Routine) is a special function that responds immediately to hardware events, like a button press or a timer reaching zero.
ISR stops whatever the system was doing to quickly handle the event.

→ An IST (Interrupt service Task) comes into play after an ISR has quickly handled the immediate needs of an interrupt. By offloading the more time-consuming work to the IST, the ISR remains short, allowing the system to handle other interrupts more efficiently.

→ A Task is a unit of work that the operating system schedules to run continuosly or periodically in the background. Unlike ISRs, Tasks are not triggered by hardware events but instead are managed by the system.

5. Explain about inter process communication

Inter process communication (IPC) is a way for different programs, called processes, to talk to each other and share information. Imagine you have two friends who need to share messages to work together on a project. Similarly, in a computer, processes need to communicate to perform tasks efficiently. IPC helps these processes exchange data, coordinate actions, and use shared resource without interfering with each other.

Types of Inter process communication.

1. Pipes:

Pipes are like a one-way tube that connects two processes. One process sends data into the pipe, and the other process reads the data from the other end. It's a simple way for two related processes to

to communicate directly.

2. Message Queues :

Message Queues act like a line of messages waiting to be read. processes can send messages to the queue and other processes can read them in the order they were sent. This allows multiple processes to communicate asynchronously, meaning they don't have to wait for each other.

Example : Imagine a mailbox where one person drops letters in, and another person collects and reads them later.

3. Shared Memory

Shared Memory allows multiple processes to access the same memory space. Instead of sending data back and forth, processes can read from and write to this common area directly, making communication very fast.

Example : picture a white board in a classroom where all students can write and read notes simultaneously.

4. Semaphores :-

semaphores are like traffic signals that control access to shared resources. They help prevent conflicts when multiple processes try to use the same resource at the same time by signaling when a process can use the resource.

Example ; Think of a semaphore as a stop light that tells cars when they can go or need to stop to avoid accidents.

5. Sockets :-

Sockets enable communication between processes over a network or within the same computer. They work like phones lines, allowing processes to send and

receive data to and from each other, whether they are on the same device or different ones connected through the internet.

Example: Imagine two friends talking on the phone, sending and receive messages in real-time.

Why IPC is Important

IPC is essential because it allows different programs to work together smoothly. For example, a web browser (one process) might communicate with a printing service (another process) to print a web page. Without IPC, these process would have a hard time sharing information and coordinating their actions, making the computer system less efficient.

6. Explain the concept of semaphores.

Semaphores are synchronization tools used in programming to control access to control access to shared resources in a concurrent system. They are particularly important in real-time operating system (RTOS) and multithreaded applications, where multiple threads or processes may need to access the same resource simultaneously.

Key characteristics of semaphores

1. Counter:

A semaphores maintains an integer value (the counter) that represents the number of available resources.

The value can be:

Positive: Indicates the number of available resources

Zero: Indicates that no resources are available resour

Negative: Indicates the number of processes waiting for a resource.

2. Two Types:

counting semaphore: This type can have any non-negative

value and is used when the number of available resources can be more than one (eg. a limited number of identical resources).

Binary semaphore:

This type can only take values 0 or 1. It is used to implement mutual exclusion, ensuring that only one thread or process can access a resources at a time.

How semaphores work:

Intialization: A semaphore is intialized to a certain value, representing the number of available resources.

Wait operation (P or down):

When a thread wants to access a resource, it performs a wait operation.

Signal operation (V or up):

When a thread has finished using the resource, it performs a signal operation.

Example of semaphore Usage

1. Initialization:

A binary semaphore is intialized to 1, indicating that the printer is available.

2. Thread Requests printer:

Thread A wants to print and calls the wait operation on the semaphore. The semaphore value is decremented from 1 to 0, allowing Thread A to proceed with printing

3. Thread finishes printing:

After Thread A finishes printing, it calls the signal operation, incrementing the semaphore value back to 1. The printer is now available for other threads.

4. Thread B requests printer:

If thread B request the printer while Thread A is

still printing, it will call the wait operation and be blocked until Thread A releases the printer.

Benefits of using semaphores

1. Mutual Exclusion :

semaphores help prevent race conditions by ensuring that only one thread can access a critical section of code or a resource at a time.

2. Resource Management :

They allow for effective management of resources by controlling how many threads can access a particular resource simultaneously.

3. Thread synchronization :

semaphores can be used to synchronize the execution of threads, making sure they operate in the correct order.

semaphores are a crucial synchronization mechanism in concurrent programming. They help manage access to shared resources, ensuring that applications can operate safely and efficiently without running into issues like race conditions or resources contention.

understanding semaphores is fundamental for developing robust multithreaded and real-time applications.

7. Explain the concept of disabling and enabling functions.

The concepts of disabling and enabling functions primarily relate to managing interrupts in embedded systems and real-time operating system (RTOS). These functions allow developers to control the responsiveness of their applications, ensuring that critical sections of code can execute without interruption. Here's an over view of these concepts :

1. Disabling Functions :

Disabling interrupts means preventing the processor from responding to interrupt requests. This is essential when a critical section of code needs to execute without being interrupted by external events, which could lead to inconsistent states or data corruption.

When interrupts are disabled, the processor will ignore all incoming interrupts until they are re-enabled. This is commonly used in situations where data needs to be safely accessed or modified, such as in shared resource management.

2. Enabling Functions :

Enabling interrupts allows the processor to respond to interrupt requests again. Once interrupts are enabled the system can handle events as they occur, which is crucial for real-time responsiveness in applications.

Example usage :

```
1. // Disabling interrupts :
   - disable - irq();
// critical section : accessing shared resources.
   shared - variable ++ ;
// Re-enable interrupts
   - enable - irq();
```

Conclusion :

Disabling and enabling functions are essential for ensuring the integrity of data and the proper operation of critical sections in embedded systems and RTOS applications. By controlling interrupt handling, developers can effectively manage shared resources, maintain system stability, and ensure that their applications respond appropriately to events.

8. Explain the concept of pipe and socket functions.

Pipes : pipes are simpler and more limited in scope, as they are designed for communication within the same machine.

-> pipes are more suitable for communication b/w processes within a single machine and might not scale well to distributed environments.

-> pipes are typically unidirectional, meaning data flows in one direction from the writer process to the reader process. For bidirectional communication, two pipes are usually needed.

-> pipes are typically created by one process and can be passed to another process, allowing for communication between them.

-> pipes are limited to communication between processes within the same machine, making them suitable for local interprocess communication.

-> pipes are used for communication b/w processes within the same system. They provide a way to send data from one process to another, typically in a producer - consumer relationship. pipes can be either unnamed (created using the pipe() system call) or named (created using the mkfifo() system call).

Sockets :-

-> Sockets are used for communication b/w processes over a network, either on the same machine (using unix domain sockets) or across different machines. They are commonly used for client - server communication. Where one process acts as a server listening for incoming connections, and other processes act as clients connecting to the server.

→ Socket provide a communication channel over a network enabling processes on different machines to exchange data.

→ Sockets can provide communication b/w processes on different machines, making them suitable for networked applications.

→ Sockets provide more flexibility as they can be used for communication across different machines can be implemented using Darious protocols (TCP, UDP, etc).

9. Briefly explain remote procedure call functions.

Remote procedure call (RPC) is a powerful communication method that enables a program on one computer to execute a procedure on another computer within a network. This is particularly useful in distributed computing systems, where applications are designed to run across multiple servers or machines.

How RPC works:

1. Client Request: The client application initiates an RPC by calling a function that appears to be local. However, this call is intercepted by a client stub, which prepares a request packet containing the function name, parameters, and additional meta data.

2. Data Serialization: The client stub serializes (or marshals) the parameters into a format that can be transmitted over the network. This step ensures compatibility between different machine architectures or data formats.

3. Network communication: The serialized request is sent over the network to the remote server where the actual procedure is located. The server stub receives the request and deserializes (or unmarshals) the parameters.

4. Function Execution : The server stub calls the requested procedure with the deserialized parameters. The procedure executes as if it were called locally on the server.

5. Response : once the procedure completes, the server stub serializes the result and sends it back to the client. The client stub receives and response, allowing the client application to access the returned data seamlessly.

6. Transparency : one of the key features of RPC is transparency. The client application perceives the RPC function as a local call, without needing to manage the underlying network details.

Advantages of RPC :

1. Simplicity
2. Modularity
3. Transparency.

Disadvantages of RPC :

1. Network dependency
2. platform dependency
3. complexity.

10.

What is inter-process communication explain.

Refer Question - 5

11. Write short notes on queues and mailboxes.

Queues :

A queue in embedded systems is a simple way to hold and manage a sequence of data, usually on a First-in-First-Out (FIFO) basics. This means the first item that goes in is the first item to come out.

Feature	Queue	Mailbox
Purpose	Data Storage in FIFO order	Message passing between tasks
Data Type	Simple data items	Full messages or Commands.
order	First-In-First-Out	order depends on mailbox implementation

uses of Queues :-

→ Queues are used to store data temporarily until another part of the system is ready to use it.

→ organize tasks or message so that they are handled in order.

Mail boxes :-

A mailbox is a way to send message between tasks unlike a queue that only handles simple data, a mail box can pass larger or more complex message from one part of a system to another.

use of Mailboxes :-

→ Tasks need to communicate by sending messages or signals.

→ you need to send a mix of data, like a status update or command.

conclusion :-

→ Queues are like storage bins for data that must be processed in order. They are ideal for buffering and ordering simple data.

→ Mailboxes are used for communication between tasks. They help one task send instructions to another task, making them great for handling commands or more complex information.

12. Explain briefly with an example

a) Inter Process communication

b) process

c) Thread.

Refer Questions, – 1, 2, 5.